

Soaring / Thermalling — Current State & Development Directions

Scope: UAVXArmQ/src/soar/soar.{c,h}, soar/ekf.{c,h}, soar/MatrixMath.{c,h} plus the integration points in auto.c, control.c, nav.c, mixer.c, telem.c. Based on ArduSoar (Peter Braswell / Samuel Tabor), CSz/Tabor Gaussian-updraft thermal-centre EKF.

1. What exists (the algorithm is fully present)

- **Vario:** UpdateVario() (soar.c:237). With USE_NETTO it computes a TE-compensated + optionally Netto/polar-corrected vario (CorrectNettoRate, soar.c:194); without it, Vario = VarioFilt = ROC. Driven only for FW (control.c:352).
- **Thermal state EKF** (soar/ekf.c): 4-state {strength, radius, x, y} tracking a Gaussian updraft. InitThermalling() (soar.c:111) resets it with the thermal placed ahead of the aircraft (THERMAL_DIST_AHEAD_M). UpdateThermalEstimate() (soar.c:146) feeds vario as the measurement and logs telemetry into SoaringTune.
- **State machine (cross-country glide strategy):**
 - DoGliderStuff() (auto.c:232): the FW soaring supervisor.
 - SuppressThrottle() (soar.c:81): cuts throttle above AltCutoffM.
 - CommenceThermalling() (soar.c:96): after CRUISE_MIN_MS, vario>min, in alt band → enter UsingThermal.
 - ResumeGlide() (soar.c:103): on thermal timeout / out of band / thermal dead → back to JustGliding.
 - MacCready() (soar.c:45) and InAltitudeBand() (soar.c:75) helpers.
- **Throttle cut implementation:** mixer.c:199 — ThrottleSuppressed ? 0.0f : ... already zeroes motor throttle (note: this is the MR mixer line — see §3).

2. The problem — the whole path is dead-gated

The FW soaring/glide strategy is unreachable. F.Glide is the master gate and it is **never set to true anywhere**:

- Only assignments in the codebase are F.Glide = false (auto.c:276, :296, and nav.c:114 via ZeroNavCorrections). No F.Glide = true exists.
- Gate consumers: control.c:251 (F.Glide && F.NavigationEnabled → BoostClimb/thermal handling) and auto.c:481 ((CAT_FW) && F.Glide → JustGliding).

Consequences: - NavState can never reach JustGliding (the soaring entry state) via auto.c:481, so DoGliderStuff() (JustGliding/UsingThermal cases at auto.c:352-355) never runs. - BoostClimb is also unreachable from the soaring supervisor (it can be entered only inside DoGliderStuff), so DoROCControl / the “rapid climb then motor-off glide” strategy the AGENTS.md TODO describes is latent.

Also dead / never invoked from outside: - InitCruising() (soar.c:141) — no caller. - SuppressThrottle() (soar.c:81) — defined, exported, **no caller**. So ThrottleSuppressed is never set → mixer.c:199 always takes the live throttle branch. - SoaringTune telemetry (SendSoaringPacket, telem.c:473) is commented out. - F.Soaring is a real flags bit (main.h:160) but its soaring/thermal-exempt logic in CheckAltHoldAlarm (control.c:100-103) is likewise unreachable while glide is off (irrelevant for the MR ExcessLift path, fine for FW).

3. Key integration remarks / risks

1. **Throttle-cut lives in the MR mixer line.** `mixer.c:199` is the multirotor throttle output (`DesiredThrottle + AltHoldThrComp`). For FW the servo/throttle output is handled elsewhere (fixed-wing mixer). Confirm the FW throttle path also honors `ThrottleSuppressed`, else soaring cuts the wrong output. The comment `// zzz spdHgt->reset_pitch_I()` and the disabled wind-correction (`soar.c:157-166`) mark unfinished integration.
2. **Wind correction is disabled** (`soar.c:157` block commented out). The thermal-center EKF accumulates upstream drift while circling. `Wind.Est` is produced by `wind/wind.c:35 EstimateWind()` (uses `GPSKF.Vel` when available), and `F.WindEstValid` already exists — the re-enable is the commented 5 lines feeding `dx_w/dy_w`.
3. **UpdateVario is FW-gated** (`control.c:352`, `pAFTTypeCategory == CAT_FW`). Fine for the FW glider strategy; intentional.
4. **Rigid constants.** Everything is `#defined` in `soar.h` (thermal min sink, dist-ahead, min thermal/cruise times, alt band, strengths) with no parameterization. The `AGENTS.md` TODO wants tunability; these should move to the param table if total-energy/Netto is ever activated.
5. **GPS-KF dependency.** `UpdateThermalEstimate` uses `Nav.C` positions; if GPS KF / nav updates are not running, `F.NewNavUpdate` gating must be verified — `DoNavigation` (`auto.c:334`) only calls into the soar/glide machinery under `F.NewNavUpdate`.
6. **ThermalOK + EKMeasurementpredandjacobian radius coupling** — the Kalman state and the expected-measurement model share `ekf.X`. Radius prior `INIT_RADIUS_COVARIANCE` is large (2500) vs `THERMAL_Q2`; initial convergence behaviour is untested (no sim harness for soaring).

4. Development directions (prioritised)

Phase 0 — bring it back alive (enable the gate): 1. Define how `F.Glide` becomes true. Options: (a) a config bit / param `SoaringEnabled` (GCS enum) gated to `CAT_FW`; (b) automatic on `WPNav+HoldingStation` for FW. Recommend (a) for safety — soaring must never engage on a multirotor or by surprise. 2. Wire `InitCruising()` at the transition into `JustGliding` and call `SuppressThrottle()` in the FW control loop so mixer/servo throttle honours `ThrottleSuppressed`. 3. Verify the **FW throttle-output path** cuts on `ThrottleSuppressed` (not just `mixer.c:199`). 4. Re-enable `SoaringTune` telemetry (uncomment `SendSoaringPacket`) so the GCS can display vario/thermal strength/dx,dy and confirm the EKF is converging.

Phase 1 — harden: 5. Re-enable wind correction (uncomment `soar.c:157-166`) — coordinates the circling track against cross-wind; without it the centering EKF drifts. 6. Add a soaring telemetry trace / GCS panel (vario, thermal strength, radius, state) and a sim check (reuse `tests/test_robustness_sim` pattern) for the thermal EKF before flight.

Phase 2 — parameterize & tune: 7. Promote the `soar.h` constants to the param table (min sink, dist-ahead, thermal/cruise times, alt band, KF `THERMAL_Q1/Q2/R`). 8. Tune `THERMAL_Min,R,Q` against real/logged vario; the flat `Vario=ROC` mode (no `USE_NETTO`) cannot see lift until the aircraft is already climbing — a `TE/Netto` vario is the proper long-term input. 9. THEN revisit the `BoostClimb/ROC` strategy (`AGENTS.md`) once the base thermal-centring loop is demonstrably converging.

Do not attempt Phase 2 tuning before Phase 0 (the code cannot currently run). The quickest high-value action is Enable-Gate + telemetry (Phase 0, #1-4).